

Notebook

February 8, 2026

1 Montgomery Curves and Insecure Twists

We've looked into what elliptic curves are and which benefits they bring to the cryptographic table in the previous exercises, but we focused only on the short form of Weierstrass curve equation. There are several reasons for that:

- it is easier to comprehend
- group laws are easy to define

There are, however, other equations defining certain families of elliptic curves, that may be a bit harder to understand, but have undeniable benefits. One of such families is **Montgomery curves**. They are just one of wonderful creations by Dr. Peter Lawrence Montgomery, who has had a huge impact on modern cryptography. Unfortunately, he passed away in 2020. You can read his obituary that touches on his work here: [Peter Lawrence Montgomery](#)

1.1 Montgomery Curves

Montgomery curves are defined by an equation over a field F : $M_{A,B} : By^2 = x^3 + Ax^2 + x$, where $A, B \in F$ and $B(A^2 - 4) \neq 0$. We can rewrite any Montgomery curve into Weierstrass short form. To do this we first need to divide each term of equation by B^3 and substitute the variables x and y with $u = \frac{x}{B}$ and $v = \frac{y}{B}$ respectively. The result will be a new equation:

$$v^2 = u^3 + \frac{A}{B}u^2 + \frac{1}{B^2}u$$

Then we can transform this into the short form by substituting u with $t - \frac{A}{3B}$:

$$v^2 = \left(t - \frac{A}{3B}\right)^3 + \frac{A}{B}\left(t - \frac{A}{3B}\right)^2 + \frac{1}{B^2}\left(t - \frac{A}{3B}\right)$$

which turns into

$$v^2 = t^3 + \left(\frac{3 - A^2}{3B^2}\right)t + \left(\frac{2A^3 - 9A}{27B^3}\right)$$

So the mapping is as follows:

$$(x, y) \mapsto (t, v) = \left(\frac{x}{B} + \frac{A}{3B}, \frac{y}{B}\right), a = \frac{3 - A^2}{3B^2}, b = \frac{2A^3 - 9A}{27B^3}$$

While we can map any curve to Weierstrass form, not all curves are meant to be Montgomery curves. Curves can be converted to Montgomery form iff (if and only if)

the initial curve $E_{a,b}$ has the following characteristics: 1. Order of $E_{a,b}$ is divisible by four 2. $z^3 + az + b = 0$ has at least one root $\alpha \in F$, where F is the field over which the curve is defined 3. $3\alpha^2 + a$ is a quadratic residue in F

We can see that Montgomery curves are a subset of elliptic curves. What's their advantage? Well, actually, scalar multiplication on Montgomery curves can be performed by an efficient algorithm called Montgomery ladder:

```
function ladder(u, k):
    u2, w2 := (1, 0)
    u3, w3 := (u, 1)
    for i in reverse(range(bitlen(p))):
        b := 1 & (k >> i)
        u2, u3 := cswap(u2, u3, b)
        w2, w3 := cswap(w2, w3, b)
        u3, w3 := ((u2*u3 - w2*w3)^2,
                    u * (u2*w3 - w2*u3)^2)
        u2, w2 := ((u2^2 - w2^2)^2,
                    4*u2*w2 * (u2^2 + A*u2*w2 + w2^2))
        u2, u3 := cswap(u2, u3, b)
        w2, w3 := cswap(w2, w3, b)
    return u2 * w2^(p-2)
```

Cryptopals say you're not supposed to understand this. Frankly, I agree. But if you really want to understand most of the math and ideas behind this short but fascinating algorithm, read [Montgomery Curves and Montgomery ladder](#). There is also lots of general elliptic curve theory. However, there are some features of this algorithm I must point out:

- Montgomery ladder is a single-coordinate ladder. What this means is that the ladder computes u for $k * P$, given just u of P . Since it is only using u , both (u, v) and $(u, -v)$ are represented by one value of u .
- w used in the algorithm is a coordinate used in alternative point representation. Points on elliptic curves can be represented in so-called projective coordinates. For example, for a short Weierstrass form $y^2 = x^3 + ax + b$ the equation using projective coordinates would be $Y^2Z = X^3 + aXZ^2 + bZ^3$, so the coordinates are $(X : Y : Z)$. What's more $(X : Y : Z)$ and $(\lambda X : \lambda Y : \lambda Z)$ are equivalent if $\lambda \neq 0$. And we can always compute equivalent $(x : y)$ pair as $(X/Z : Y/Z)$ (if $Z \neq 0$). The point at infinity is usually represented as $(0 : 1 : 0)$. Projective coordinates can make computation more efficient. Division (or computing the inverse of a value) consumes lots of resources. Various sources estimate it to be equivalent to 80~100 multiplications, so using division abundantly is undesirable. Montgomery ladder leverages projective coordinates to defer division until the last moment. You may remember, that $a^{(p-2)} = a^{-1} \bmod p$, so we compute the affine coordinate (usual representation) u value just before we return.
- $\text{cswap}(a,b,c)$ means swap a and b if c . This operation can be implemented without branching:

```
t := a xor b
```

```

m := neg c
t := t and m
a := a xor t
b := b xor t

```

Imagine that this is executing under some 32-bit architecture. Since the values we'd like to swap probably can't fit into 32 bits, we are operating on pointers to structures in memory. So a and b are pointers. First, we compute xor of a and b , then we negate c . If c is 0, then m is 0. If c is 1, it m is 0xffffffff. Then we and t and m . t will either stay the same (if m is 0xffffffff) or become 0. Then we xor a and b with either 0 (which make no change) or $(a \text{ xor } b)$, which swaps the values. Why is it important to avoid branching constructions during private key operations? Branching constructions are particularly vulnerable to side channel attacks (different paths and instructions). Those can be based on timing, power or electromagnetic radiation analysis. `cswap` is better protected against them.

- The algorithm uses so-called differential addition. We can add two points P and Q only if we know the difference between them. The algorithm cleverly uses addition and doubling, preserving the difference between the two values while computing the end result.
- The algorithm gracefully handles corner cases (adding point at infinity to other points, etc) without any branching.

1.2 Twist

Remember how we used the poisonous points to trick Weierstrass form doubling and addition algorithms into revealing the secret key? Since we only have one coordinate to work with, that particular method won't do. But all is not lost.

When we were searching for points on the curves with nicely factorizable orders, we were checking if for a particular x the resulting y^2 was a quadratic residue in F . After all, if it was not a quadratic residue, then we couldn't compute roots. Let me show you something. Imagine we have a regular elliptic curve in Weierstrass short form over field F :

$$y^2 = x^3 + ax + b$$

What happens if we multiply the left half of it by a number $d \neq 0$?

$$dy^2 = x^3 + ax + b$$

If d is a quadratic residue and m is one of its roots, then we could substitute $my \mapsto y'$ and we'd get:

$$y'^2 = x^3 + ax + b$$

So same equation, effectively same curve. It would only change points' positions, not relations between them. Now, what happens if d is not a quadratic residue? Those values of x that were producing points, since y^2 was a quadratic residue, can't produce points on this new curve. Now all the other values of x do. This curve is called a quadratic twist of the original curve. So the values of x that turn the right half of the equation into a quadratic non-residue produce another curve. As you could imagine,

the same is true for Montgomery form, since we have v^2 on the left. And if we were to call Montgomery ladder with the value of u that evaluates a quadratic non-residue, we would compute the u -coordinate of a point on the quadratic twist. So now we can once again submit a point on an alternative curve, but for it to be useful, we need to know the order of this curve. So let's compute it:

1. We know, that there are p different elements in the field F_p . One of them is special case - 0, so we have $p - 1$ regular values and 0.
2. The right part of the equation evaluates to all values in F_p . If it is 0 the Point exists both on the original curve and on the quadratic twist. For all other values in F_p the right side of the equation is either a quadratic residue, and there are 2 points on the original curve, or it is a quadratic non-residue and there are two points on the twist.
3. Both curves have the point at infinity. So the total number of points on both curves is $2 + 2(p - 1) + 2 = 2(p + 1)$. We can compute the number of points on the quadratic twist by subtracting the order of the original curve from $2(p + 1)$.

You should also know, that the order of the original curve is $\#E(F) = p + 1 - t$, where t called the Trace of Frobenius and according to Hasse's theorem $|t| \leq 2\sqrt{p}$.

1.3 Task

We are going to borrow curve parameters from Cryptopals (since their curve is awesome). The initial Weierstrass short form is:

$$y^2 = x^3 - 95051x + 11279326 \text{ over } GF(233970423115425145524320034830162017933)$$

One of possible basepoints is (182, 85518893674295321206118380980485522083). The order of the curve is

$$233970423115425145498902418297807005944 = 2^3 * 29246302889428143187362802287225875743$$

Converting a point to Montgomery is extremely simple (once again, thanks Cryptopals):

$$(u, v) = (x - 178, y)$$

Montgomery form is

$$v^2 = u^3 + 534u^2 + u$$

The server's secret key $k = 0 \bmod 1571528514013$. It is possible to solve DLP in this subgroup in python, but it takes too long (took several hours on a single thread on my laptop). You will, however, have to use Pollard. You can use it even with Montgomery. Use the rho variant.

```
//Single jump:
j=hash(point)
xT=(xT*j)%subgroup_order
yT=ladder(yT,j)
```

Hint: j should never be 0 or a multiple of subgroup order, since the result of Montgomery ladder will always be 0.

Now, your goal is once again to find the server's secret key. To achieve this:

1. Compute the order of the quadratic twist
2. Factorize the twist order into (4, 11, 107, 197, 1621, 105143, 405373, 2323367, 1571528514013)
3. Find a point on the quadratic twist (just find a u that turns the right side of the equation into a quadratic non-residue)
4. Submit points from subgroups of the quadratic twist (same as in the previous exercise)
5. CRT, but you will have to do this gradually. Each value u represents 2 points, so when you find some r for a subgroup of order d , there can actually be two possible remainders: r and $d-r$. If you were to collect all remainders independently, then you'd have to try all possible combinations. This is unnecessarily hard. Let's say you have (r_1, d_1) and (r_2, d_2) . You can compute the 4 possible remainders modulo $d_1 d_2$, then check which ones of these 4 are viable, by sending points confined to twist's subgroup of order $d_1 d_2$. You'll reduce possibilities to 2. Repeat the process for d_i , where $i > 2$
6. Profit

P.S. If you want to look some more into elliptic curves and are interested in learning about standard curves, [Safecurves](#) is a good place to start. If you want to implement a curve, say to study it (please **DON'T ROLL YOUR OWN CRYPTO**) go to [Explicit Formulas Database](#).

```
[ ]: def create_ladder(p,A):
    bitlen_p=0
    p1=p
    while p1!=0:
        p1>>=1
        bitlen_p+=1

    def ladder(u,k):
        nonlocal p,bitlen_p,A
        (u2,w2)=(1,0)
        (u3,w3)=(u,1)
        for i in range(bitlen_p):
            b=(k>>(bitlen_p-1-i))&1
            if b==1:
                u2,u3,w2,w3=u3,u2,w3,w2
                u3,w3=pow((u2*u3-w2*w3)%p,2,p) , (u*pow(u2*w3-w2*u3,2,p))%p
                u2,w2= pow(pow(u2,2,p)-pow(w2,2,p),2,p) ,
                ↪ (4*u2*(w2*(pow(u2,2,p)+A*((u2*w2)%p)+pow(w2,2,p))%p)%p
            if b==1:
                u2,u3,w2,w3=u3,u2,w3,w2
        return (u2*pow(w2,p-2,p))%p
    return ladder
```

```

[ ]: import socket
import re
class VulnServerClient:
    def __init__(self, show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1349))

    def recv_until(self, symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data

    def getChallenge(self, show=True):
        """Get the challenge paramteres from the server"""
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server,
again.')
```

```

            return (None,None)
        if show:
            print (data)

        p=int(re.search(r'(?<=GF\()\d+(?=\))',data).group(0))
        group_order=int(re.search(r'(?<=order )\d+',data).group(0))
        A=int(re.search(r'(?<=u\^3\+)(-\?\d+u)',data).group(0)[: -1])
        return (p,A,group_order)

    def checkSolution(self,k, show=True):
        """Check if you've found the right k (shows the flag if it is,
right and show==True)"""
        self.s.sendall(('check '+str(k)+'\n').encode())
        data=self.recv_until(b'\n')
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server,
again.')
```

```

            return None
        if show:
            print (data)

```

```

        if data.find('flag')!=-1:
            return True
        else:
            data=self.recv_until(b'>')
            try:
                data=data.decode()
            except UnicodeDecodeError:
                print ('Error decoding unicode. Try connecting to_
↪server again.')
                return None
            if show:
                print (data)
            return False

    def getPointMultiple(self,u, show=True):
        """Get k*<your_point> from the server"""
        self.s.sendall(('mult '+str(u)+'\n').encode())
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server_
↪again.')
            return None
        if show:
            print (data)

        point=re.search(r'\d+',data).group(0)

        return int(point)

    def __del__(self):
        self.s.close()

try:
    from gmpy2 import invert
except ImportError:
    try:
        from Crypto.Util.number import inverse as invert
    except ImportError:
        print ("You need to install either gmpy2:\nsudo apt install_
↪python-gmpy2\nor pycryptodome:\npython3 -m pip install_
↪pycryptodome")
        raise Exception
def crt(remainders,modules,M):
    result = 0

```

```
    for (a, b) in zip(remainders,modules):
        result = (result+a*((M)//b)*invert((M)//b, b)) % M
    return result
```

```
[ ]: vs=VulnServerClient()
      p,A,group_order=vs.getChallenge()
      ladder=create_ladder(p,A)
      print (ladder(4,group_order),ladder(4,group_order-1))
      print (vs.getPointMultiple(4,show=False))
```

```
[ ]: vs.checkSolution(1)
```

```
[ ]:
```